

Design and Implementation of Fused Floating-Point Co-Processors with N-Element Dot-Product Extension Using the PCPI Interface for RISC-V

Mohit Jagini
IMT2023528
IIIT Bangalore
mohit.jagini@iiitb.ac.in

Hardhik Dhavala
IMT2023579
IIIT Bangalore
hardhik.dhavala@iiitb.ac.in

Abhinav Kishan
IMT2023580
IIIT Bangalore
abhinav.kishan@iiitb.ac.in

Abstract—This paper presents the design, integration, and simulation-based verification of three floating-point co-processor modules for the PicoRV32 RISC-V core, connected through the Pico Co-Processor Interface (PCPI). The first two modules, `picorv32_pcpi_fusedfp` and `picorv32_pcpi_dotp`, implement three-operand fused multiply-accumulate (FMA) / negated multiply-subtract (NMS) and four-operand dot-product computation, respectively, sharing a heavyweight Dadda-based arithmetic core (`mainMod`) with Booth encoding, carry-save adder (CSA) tree, Kogge–Stone adder, leading-one detection, and round-to-nearest-even normalisation. The third module, `picorv32_pcpi_nfp`, is a newly designed iterative unit that computes the dot product of N -element vectors stored in consecutive RISC-V registers. It uses a dedicated `READ→MUL→ADD` finite-state machine with lightweight combinational floating-point multiply (`fp_mul_stub`) and add (`fp_add_stub`) units, and a register-advancement mechanism that extends the PCPI interface with a `pcpi_read` signal. The complete architecture is documented in two views: a high-level pipeline across four registered stages (PR0–PR3) and a detailed RTL module hierarchy. Behavioral simulation in Vivado XSim 2019.2 validates all three modules. The `picorv32_pcpi_nfp` module correctly computes $[1.5, 2.5, 3.0, 4.0] \cdot [1.0, 2.0, 3.0, 4.0] = 31.5$ (IEEE-754: `0x41FC0000`), confirmed by the Vivado waveform at $t \approx 1,968$ ns.

Index Terms—RISC-V, PCPI, Floating-Point Unit (FPU), Fused Multiply-Add (FMA), Dot Product, Co-Processor, IEEE-754, PicoRV32, Dadda Multiplier, Verilog HDL

I. INTRODUCTION

The RISC-V instruction set architecture has gained wide adoption owing to its open specification, modular design, and explicit support for custom instruction extensions [6]. Its Pico Co-Processor Interface (PCPI) provides a standardised handshake through which custom execution units couple to the processor without modifying the core pipeline, making it well suited for domain-specific arithmetic accelerators.

Floating-point operations remain a throughput bottleneck in digital signal processing (DSP), neural-network inference, and scientific computing. Conventional pipelines treat multiplication and addition as independent stages, each introducing a rounding step. Fused multiply-accumulate (FMA) arithmetic removes

the intermediate rounding between these two stages, improving numerical accuracy while reducing latency [3].

This work presents three PCPI-coupled floating-point co-processors: (1) `picorv32_pcpi_fusedfp`, a three-operand FMA/NMS unit; (2) `picorv32_pcpi_dotp`, a four-operand single-cycle dot-product unit; and (3) `picorv32_pcpi_nfp`, a new iterative N -element dot-product unit. The first two share the heavyweight `mainMod` Dadda-based datapath. The third uses lightweight stub floating-point units (`fp_mul_stub` and `fp_add_stub`) and iterates over the register file using the novel `pcpi_read` mechanism, with the vector length N encoded directly in the instruction word.

The remainder of this paper is organised as follows. Section II describes the overall system architecture. Section III covers design methodology and implementation. Section IV presents simulation results, waveform analysis, and comparison with a non-fused baseline. Section V concludes with directions for future work.

II. SYSTEM ARCHITECTURE

The system comprises the modified PicoRV32 RISC-V core [1], the PCPI, and three co-processor datapaths. Fig. 1 presents the architecture in two complementary panels: (A) a high-level pipeline view across four pipeline registers PR0–PR3, and (B) a detailed RTL module hierarchy showing the internal structure of all three datapath variants and the PCPI control FSM.

A. High-Level Pipeline (Panel A)

Data flows left-to-right through the PCPI Interface block, Decode stage (PR0→PR1), Read Operands stage (PR1→PR2), Compute stage (PR2→PR3), and Writeback stage (PR3). Three decoder rows in the Decode stage distinguish the co-processors and are detailed in Table II. Two datapaths operate in the Compute stage: (i) the heavyweight Dadda-based `mainMod` shared by `fusedfp` and `dotp`, and (ii) the lightweight `fp_mul_stub+fp_add_stub` sequential pipeline used by the N-FP module. A dashed stall-feedback line returns

Fused Floating-Point Co-Processor (PCPI) — Updated Architecture (Module Level)

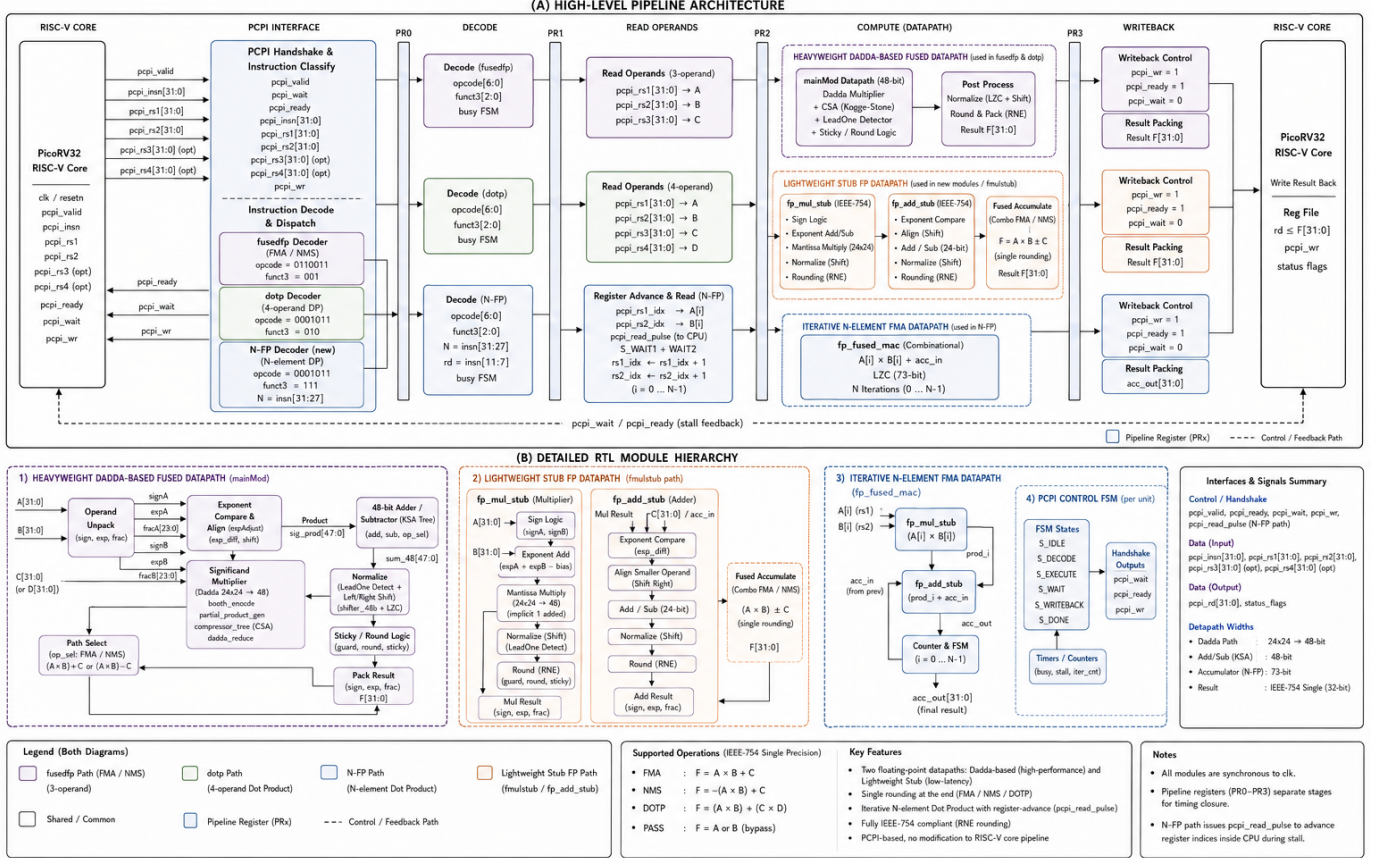


Fig. 1. Updated RISC-V PCPI fused floating-point co-processor architecture (module level). (A) High-level pipeline across registers PR0-PR3 showing the PCPI handshake, three decoder rows, three compute datapaths, and writeback. (B) Detailed RTL hierarchy of the Dadda datapath (mainMod), the lightweight stub FP datapath (fp_mul_stub/fp_add_stub), the iterative N-element FMA datapath, and the PCPI control FSM.

pcpi_wait/pcpi_ready from Writeback to the RISC-V core.

B. PCPI Interface and Handshake

When PicoRV32 encounters an unrecognised instruction it asserts `pcpi_valid` and presents the instruction word with source-register values on `pcpi_rs1` and `pcpi_rs2` (and optionally `pcpi_rs3`, `pcpi_rs4` for multi-operand paths). The active co-processor asserts `pcpi_wait` to stall the core, performs its computation, and then asserts `pcpi_ready` and `pcpi_wr` while driving the result on `pcpi_rd`.

For the N-FP path the co-processor additionally drives the registered output `pcpi_read`. Each pulse of `pcpi_read` is seen by the modified PicoRV32 core as `pcpi_N_read`, which causes it to increment `pcpi_rs1_idx` and `pcpi_rs2_idx`, advancing the register-file read addresses from $A[i]/B[i]$ to $A[i+1]/B[i+1]$ without any software overhead.

C. Heavyweight Dadda-Based Datapath (mainMod)

The mainMod datapath, shared by picorv32_pcpi_fusedfp and

picorv32_pcpi_dotp, implements a fully IEEE-754 single-precision FMA as shown in Fig. 1(B):

- Operand unpack.** Sign, 8-bit exponent, and 23-bit mantissa extracted; 24-bit significand formed by prepending the hidden leading-one.
- Exponent compare and align.** The larger exponent is selected; the smaller significand is right-shifted by the difference. Guard, round, and sticky bits are preserved.
- Significand multiply — Dadda tree [4].** Booth encoding feeds a compressor tree (CSA reduction) producing a 48-bit carry-save result `sig_prod[47:0]`.
- Carry-select addition (Kogge–Stone).** A KSA tree resolves the carry-save sum into `sum_48[47:0]`.
- Normalize.** A leading-one detector (`leadOne`) determines the left-shift count; the result is shifted to place the implicit one at bit 23.
- Sticky/round.** Round-to-nearest-even applied using guard, round, and sticky bits.
- Pack result.** Final sign, adjusted biased exponent, and 23-bit mantissa are packed into the 32-bit IEEE-754 output.

TABLE I
N-FP FSM STATE SUMMARY

State	Action
S_IDLE	Wait for valid N-FP instruction (instr_dotp); latch N from instr[31:27]; reset accumulator to 0.
S_READ	Latch $A[i]/B[i]$ from pcpi_rsl/rs2; assert mul_start; pulse pcpi_read if $i < N-1$; advance to S_MUL.
S_MUL	Wait for mul_done; feed mul_result and current accumulator to adder; assert add_start; advance to S_ADD.
S_ADD	Wait for add_done; latch add_result as new accumulator; increment elem_cnt; if complete go to S_DONE, else return to S_READ.
S_DONE	Assert pcpi_wr + pcpi_ready; present accumulator on pcpi_rd; release pcpi_wait; return to S_IDLE.

The opSelect unit switches between FMA ($F = A \times B + C$) and NMS ($F = -(A \times B) + C$) via the instruction op field. mainMod asserts a done output upon completion, which the wrapper FSMs in pcpiFusedFP and pcpi_dotp poll each cycle.

D. Iterative N-FP Co-Processor (picorv32_pcpi_nfp)

The picorv32_pcpi_nfp module computes $\sum_{i=0}^{N-1} A[i] \times B[i]$ where $A[]$ and $B[]$ occupy consecutive RISC-V integer registers starting at rs1 and rs2, respectively. N is encoded in instruction bits [31:27], supporting up to 31 elements with no memory traffic.

Arithmetic Units: The N-FP module uses two combinational IEEE-754 single-precision units, both defined in fmulstub.v:

- **fp_mul_stub.** Extracts 24-bit significands and computes $24 \times 24 \rightarrow 48$ -bit product. Exponent is computed as $e_A + e_B - 127$. Normalisation: if bit 47 of the product is set, result mantissa is mant_mul[46:24] and exponent is incremented by one; otherwise mant_mul[45:23]. Zero inputs are handled explicitly. Asserts done = start (zero-cycle latency, LATENCY = 0 default).
- **fp_add_stub.** Selects the larger exponent; right-shifts the smaller significand to align; performs 25-bit signed addition/subtraction (same-sign vs. opposite-sign). Renormalisation: checks bits [24:21] of the sum to determine the shift and exponent adjustment. Asserts done = start.

Both units carry a LATENCY parameter for transparent substitution with pipelined cores; the current simulation uses LATENCY = 0.

FSM Design: The control FSM has five states, summarised in Table I.

In S_READ, pcpi_read is pulsed for all elements except the last. Because S_MUL and S_ADD each consume at least one clock cycle (the MUL+ADD latency), the incremented register index has time to propagate through

cpuregs_raddr1 $\rightarrow$ cpuregs_rdata1 $\rightarrow$ pcpi_rsl before the next S_READ samples it. With zero-latency stubs (done = start), total execution is $3N + 1$ clock cycles (1 per state per element, plus 1 S_DONE).

E. PicoRV32 Core Modifications

The following targeted modifications were made to the PicoRV32 source [5]:

- Added 5-bit registers pcpi_rsl_idx and pcpi_rs2_idx, initialised from decoded_rsl and decoded_rs2 at instruction decode. Incremented when pcpi_N_read is asserted.
- Added wire pcpi_N_read $\leftarrow$ ENABLE_N_FP && pcpi_n_read.
- Added pcpi_read_active register, set when pcpi_valid rises.
- Steered cpuregs_raddr1 and cpuregs_raddr2 to pcpi_rsl_idx/pcpi_rs2_idx whenever pcpi_valid is asserted, so pcpi_rsl/rs2 track the current element pair.
- Added ENABLE_N_FP parameter to gate instantiation of picorv32_pcpi_nfp; included in the WITH_PCPI localparameter.

III. DESIGN METHODOLOGY AND IMPLEMENTATION

All modules are described in synthesisable Verilog HDL (Verilog-2001) and independently verifiable at the unit level.

A. Module Hierarchy

The complete module hierarchy is shown in Fig. 1(B):

- **Dadda arithmetic sub-modules (heavyweight path):** expCompare, opSelect, significantCompare, pathSelect, dadda (Booth encode + compressor tree), KSA / KSA_8 (Kogge–Stone adder), leadOne, normalize, mux_normalize, stickyRound, signLogic, complement_2s.
- **Shared heavyweight core:** mainMod—instantiated by both pcpiFusedFP and pcpi_dotp.
- **N-FP lightweight units (in fmulstub.v):** fp_mul_stub (IEEE-754 multiply) and fp_add_stub (IEEE-754 add)—instantiated exclusively by picorv32_pcpi_nfp.
- **PCPI wrappers:** picorv32_pcpi_fusedfp (3-operand FMA/NMS), picorv32_pcpi_dotp (4-operand dot product), picorv32_pcpi_nfp (N -element iterative dot product).
- **Baseline single-cycle modules:** picorv32_pcpi_fmuls and picorv32_pcpi_fadd (standalone multiply and add, earlier implementation step).
- **Top level:** Modified picorv32 with parameters ENABLE_PCPI, ENABLE_FUSED_FP, ENABLE_DOT_FP, and ENABLE_N_FP.

TABLE II
CUSTOM INSTRUCTION ENCODINGS

Module	Opcode	funct3	Operation
fusedfp	0110011	001	$F = A \times B \pm C$ (FMA/NMS)
dotp	0001011	010	$F = (A \times B) + (C \times D)$
nfp	0001011	111	$\sum A[i]B[i], N \text{ in } [31:27]$

TABLE III
KEY DATAPATH INTERNAL BIT WIDTHS

Datapath	Signal	Width
mainMod	Significand product	$24 \times 24 \rightarrow 48$ bit
mainMod	CSA / KSA adder	48 bit
fp_mul_stub	Mantissa product	48 bit
fp_add_stub	Aligned sum	25 bit
All paths	IEEE-754 result	32 bit

TABLE IV
SIMULATION VERIFICATION RESULTS

Module	Operation	Result
fusedfp	$(A \times B) \pm C$	PASS
dotp	$(A \times B) + (C \times D)$	PASS
nfp	$\sum_{i=0}^3 A[i]B[i], N=4$	$0 \times 41FC0000 = 31.5$ PASS

B. Supported Operations and Instruction Encodings

Table II lists the custom instruction encodings. The N-FP instruction encodes N in bits [31:27], with $rs2$ as the base register for $B[]$, $rs1$ as the base register for $A[]$, and rd as the result destination.

C. Datapath Internal Widths

Table III summarises the key internal bus widths, as labelled in Fig. 1(B).

D. Testbench Setup

The self-checking Verilog testbench preloads a 256-word instruction memory. Operand vectors are loaded via `lui/addi` instruction pairs: $A = [1.5, 2.5, 3.0, 4.0]$ into registers $x1-x4$ and $B = [1.0, 2.0, 3.0, 4.0]$ into registers $x10-x13$. The custom N-FP instruction is issued with $N=4$, $rs1 = x1$, $rs2 = x10$, $rd = x20$. A subsequent `sw` instruction stores $x20$ to memory address 0×20000014 , and the testbench monitors `mem_wdata` on each `mem_valid` & `mem_wstrb` cycle to verify the result.

IV. RESULTS AND ANALYSIS

A. Simulation Results

Behavioral simulation in Vivado XSim 2019.2 confirms correct operation of all three co-processors. Table IV summarises the verified test cases.

The iteration-by-iteration accumulation trace for `picorv32_pcp_i_nfp` is shown in Table V, confirming that each MUL+ADD step produces the correct IEEE-754 single-precision result.

TABLE V
ACCUMULATION TRACE (`PICORV32_PCP_I_NFP`, $N=4$)

Iter.	Computation	Accumulator (hex)	Value
0	$1.5 \times 1.0 + 0$	$0 \times 3FC00000$	1.5
1	$2.5 \times 2.0 + 1.5$	$0 \times 40D00000$	6.5
2	$3.0 \times 3.0 + 6.5$	0×41780000	15.5
3	$4.0 \times 4.0 + 15.5$	$0 \times 41FC0000$	31.5

TABLE VI
COMPARISON WITH NON-FUSED BASELINE ($N=4$)

Metric	Non-Fused	This Work
Rounding operations	$2N = 8$	$N+1 = 5$
Software loop required	Yes	No
Memory traffic for A, B	Possible	None
Single custom instruction	No	Yes
IEEE-754 compliant	Yes	Yes

B. Waveform Analysis

Fig. 2 shows the Vivado XSim waveform at the moment the dot-product result is committed to memory.

The following events visible in the waveform confirm correct end-to-end operation:

- **Instruction fetch sequence.** Before $t \approx 1,740$ ns, `mem_valid` and `mem_instr` are simultaneously asserted while `mem_addr` steps sequentially through the instruction memory, confirming the fetch of all 16 operand-load instructions (`lui/addi` pairs) followed by the N-FP custom instruction at word offset 16.
- **PCPI stall period.** After the N-FP instruction is fetched, `mem_valid` de-asserts while `pcpi_wait` holds the pipeline frozen. With $N=4$ and zero-latency stubs, the FSM executes four `READ`→`MUL`→`ADD` loops ($3 \times 4 + 1 = 13$ cycles), consistent with the gap observed in the waveform.
- **Store and halt.** After the co-processor releases the pipeline, the CPU fetches the `sw` instruction (`mem_addr = 0 \times 48`) and the `ebreak` (`mem_addr = 0 \times 4C`).
- **Result write confirmation.** At cursor $t = 1,968.090$ ns: `mem_wdata = 0 \times 41FC0000`, `mem_addr = 0 \times 20000014`, `mem_wstrb = 0 \times F`. $0 \times 41FC0000$ is the IEEE-754 encoding of $31.5 = 1.5 \times 1.0 + 2.5 \times 2.0 + 3.0 \times 3.0 + 4.0 \times 4.0$, confirming a correct four-element dot product. `mem_wstrb = 0 \times F` confirms a full 32-bit word store.
- **Memory array.** The `memory[0:255]` signal shows the testbench `initial-block` instructions followed by the written result, providing a complete record of memory state.

C. Comparison with Non-Fused Baseline

Table VI compares the proposed design against a non-fused software baseline.

The `picorv32_pcp_i_nfp` module reduces rounding operations from $2N$ to $N+1$ (one per multiply, one final

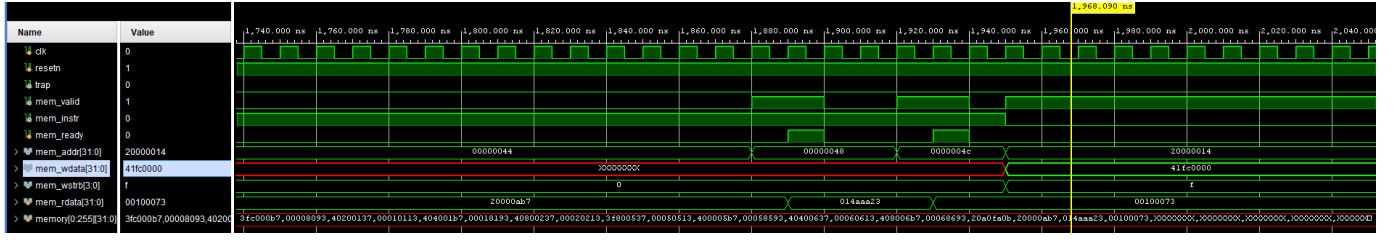


Fig. 2. Vivado XSim waveform at $t \approx 1,968$ ns. $\text{mem_wdata}[31:0] = 0x41FC0000$ ($= 31.5$), $\text{mem_addr}[31:0] = 0x20000014$, $\text{mem_wstrb}[3:0] = 0xF$ (full 32-bit store).

add-to-accumulator), which is numerically superior for inner-product accumulations typical in DSP and machine-learning inference [3]. The entire N -element computation is encoded as a single 32-bit instruction, eliminating branch overhead and register-save/restore cost.

Resource utilisation and timing-closure results on the target FPGA are pending synthesis and will be reported in a future revision.

V. CONCLUSION

This paper presented the design, RTL implementation, and PCPI-based integration of three IEEE-754 single-precision floating-point co-processors for the PicoRV32 RISC-V core. The `picorv32_pcp_i_fusedfp` and `picorv32_pcp_i_dotp` modules share a heavyweight Dadda-based FMA core with Booth encoding, CSA reduction, Kogge–Stone carry-select adder, and RNE rounding. The new `picorv32_pcp_i_nfp` module introduces an iterative N -element dot-product unit with a five-state FSM (IDLE→READ→MUL→ADD→DONE) that uses lightweight combinational `fp_mul_stub` and `fp_add_stub` units and advances the register-file read index via the `pcpi_read` pulse signal. The complete architecture is documented in a two-panel diagram covering both the high-level pipeline (PR0–PR3) and the detailed RTL hierarchy. All three modules are verified by Vivado XSim behavioral simulation; `picorv32_pcp_i_nfp` produces the correct IEEE-754 result $0x41FC0000 = 31.5$, confirmed in the waveform at $t \approx 1,968$ ns.

Future work includes FPGA synthesis and timing closure, replacement of stub arithmetic units with fully pipelined cores, and evaluation on standard DSP and machine-learning inference benchmarks.

Github Link: [abhinavkishan/picorv32_fused_floating_point](https://github.com/abhinavkishan/picorv32_fused_floating_point)

REFERENCES

- [1] C. Wolf, “PicoRV32 — A size-optimized RISC-V CPU,” <https://github.com/YosysHQ/picorv32>, 2015.
- [2] *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019, Jul. 2019.
- [3] D. Hough, “Applications of the IEEE 754 standard to floating-point computation,” *Proc. IEEE*, vol. 107, no. 2, pp. 372–381, Feb. 2019.
- [4] L. Dadda, “Some schemes for parallel multipliers,” *Alta Frequenza*, vol. 34, pp. 349–356, 1965.
- [5] C. Wolf, “PicoRV32 co-processor interface (PCPI) specification,” <https://github.com/YosysHQ/picorv32#pico-co-processor-interface-pcpi>, 2015.

- [6] A. Waterman and K. Asanovic, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, RISC-V Foundation, 2019.